

Anexo

Instalación y puesta en marcha de la PoC

Diseño y validación de una plataforma segura de APIs para integración bancaria

Autor	Antonio Martos Gavilán
Titulación	Grado de Ingeniería Informática
Área	Sistemas de Gestión del Conocimiento
Tutor	Oscar Martín Salas
Fecha	08/06/2026
Versión	1.1 - Documento de instalación final para entrega

Índice del anexo

1. Finalidad del anexo
2. Alcance de la instalación documentada
3. Requisitos previos
4. Preparación de la máquina virtual Linux
5. Obtención del producto y estructura esperada
6. Configuración de variables de entorno y secretos
7. Instalación y arranque de la PoC con Docker Compose
8. Inicialización de Kong Gateway y publicación de APIs
9. Verificación funcional de la instalación
10. Verificación de observabilidad y mensajería asíncrona
11. Parada, limpieza y reinstalación desde cero
12. Incidencias habituales y resolución
13. Checklist de evidencias para la entrega final
14. Criterios de aceptación de la instalación
15. Trazabilidad instalación-validación-evidencia
16. Limitaciones y cierre operativo
17. Referencias bibliográficas

1. Finalidad del anexo

Este anexo documenta el procedimiento de instalación y puesta en marcha de la prueba de concepto (PoC) desarrollada en el Trabajo de Fin de Grado. Su objetivo es permitir que una persona evaluadora pueda reproducir el entorno técnico en una máquina Linux controlada, levantar los servicios principales, verificar el funcionamiento de las APIs expuestas por Kong Gateway y comprobar las capacidades básicas de seguridad, mensajería y observabilidad.

El documento complementa la memoria y los anexos técnicos existentes. Mientras que los anexos de configuración describen con mayor detalle las decisiones de seguridad, contratos de API, métricas y trazabilidad, este anexo se centra en el recorrido operativo mínimo para instalar el producto y dejarlo en estado ejecutable.

Criterio de seguridad documental: no se incluyen secretos reales, identificadores privados ni tokens completos. Todos los valores sensibles se representan mediante marcadores como <TENANT_ID>, <CLIENT_ID> o <CLIENT_SECRET>.

2. Alcance de la instalación documentada

La instalación cubierta corresponde al entorno local de laboratorio utilizado para validar la arquitectura de transición de la plataforma segura de APIs. No se trata de un despliegue productivo ni de una instalación con alta disponibilidad, sino de una PoC académica reproducible basada en Docker Compose.

Tipo	Descripción
Incluido	Máquina virtual Linux Ubuntu 22.04 LTS sobre VMware Workstation Pro.
Incluido	Docker Engine y Docker Compose v2 como capa de ejecución de contenedores.
Incluido	Kong Gateway Community Edition, PostgreSQL, ClientesApi, CuentasApi, RabbitMQ, ClienteCreadoConsumer, Prometheus y Grafana.
Incluido	Configuración de variables de entorno para Microsoft Entra ID y obtención de tokens mediante client credentials.
Incluido	Pruebas de humo, comprobación de rutas, métricas, logs y acceso básico a interfaces web.
Excluido	Despliegue productivo en AKS, alta disponibilidad, WAF, gestión avanzada de secretos con Azure Key Vault y SSO de usuario final.

Tabla T.1. Alcance operativo del anexo de instalación

3. Requisitos previos

3.1. Recursos de virtualización

Elemento	Valor recomendado	Justificación
Hipervisor	VMware Workstation Pro 17 o versión equivalente	Entorno local controlado y exportable.
Sistema operativo invitado	Ubuntu 22.04 LTS 64 bits	Distribución LTS compatible con Docker Engine y .NET 8.
CPU	2 vCPU	Suficiente para las cargas de prueba académicas.

Elemento	Valor recomendado	Justificación
RAM	8 GB	Permite ejecutar <i>gateway</i> , APIs, <i>broker</i> , Prometheus y Grafana con margen.
Disco	20 GB mínimo	Espacio para sistema, imágenes Docker, logs y volúmenes.
Red	NAT	Conectividad con el host sin exposición directa a la red física.

Tabla T.2. Requisitos de la máquina virtual de laboratorio

3.2. Herramientas necesarias

Herramienta	Uso en la instalación
git	Clonar el repositorio del producto.
curl	Realizar llamadas HTTP a Kong, APIs y Entra ID.
jq	Extraer valores de respuestas JSON, especialmente tokens.
Docker Engine	Ejecutar contenedores de la PoC.
Docker Compose v2	Orquestar todos los servicios mediante docker-compose.yml.
Navegador web	Acceder a Grafana, Prometheus, RabbitMQ Management y, si procede, interfaces de administración.
.NET 8 SDK	Opcional si se compilan o ejecutan las APIs fuera de Docker.

Tabla T.3. Herramientas necesarias para instalar y verificar la PoC

3.3. Requisitos previos en Microsoft Entra ID

Antes de ejecutar las pruebas protegidas debe existir una configuración mínima en Microsoft Entra ID. Este anexo asume que la configuración ya se ha realizado según el Anexo U de configuración técnica, aun así, se resumen los elementos necesarios para que la instalación sea verificable.

Elemento	Descripción
Tenant ID	Identificador del tenant de Entra ID utilizado para emitir tokens.
Aplicación recurso	Registro que representa la API protegida. Debe disponer de <i>Application ID URI</i> , normalmente <i>api://<RESOURCE_APP_ID></i> .
Aplicación cliente	Registro que representa al consumidor machine-to-machine que solicita el <i>token</i> .
Client secret	Secreto de la aplicación cliente. Debe almacenarse solo en <i>.env</i> local y no en Git.
App roles	Roles de aplicación asignados al cliente, por ejemplo, <i>Clientes.Read.App</i> , <i>Clientes.Write.App</i> y <i>Cuentas.Read.App</i> .
Consentimiento	Consentimiento de administrador concedido para que el cliente reciba los roles en la <i>claim</i> roles.

Tabla T.4. Elementos mínimos requeridos en Entra ID

4. Preparación de la máquina virtual Linux

4.1. Actualización del sistema

Tras instalar Ubuntu 22.04 LTS en la máquina virtual, se actualiza el sistema y se incorporan las herramientas básicas de administración.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg lsb-release git jq unzip make nano
```

4.2. Instalación de Docker Engine y Docker Compose v2

La PoC se ejecuta completamente sobre contenedores, por lo que Docker debe estar instalado antes de clonar y desplegar el producto.

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

A continuación, se añade el usuario actual al grupo docker para evitar ejecutar Docker permanentemente con sudo.

```
sudo usermod -aG docker $USER
newgrp docker

docker --version
docker compose version
```

Nota: si el comando `docker` sigue devolviendo “permission denied”, cerrar la sesión SSH o reiniciar la máquina virtual para que se aplique la pertenencia al grupo `docker`.

4.3. Configuración de red y puertos

Con la VM en modo NAT, los servicios quedan accesibles desde el host mediante los puertos expuestos por Docker. La Tabla T.5 resume los puertos que deben quedar libres antes del despliegue.

Puerto	Servicio	Uso
8000	Kong Gateway	Plano de datos. Entrada principal para las llamadas a APIs.
8001	Kong Admin API	Administración local de servicios, rutas, <i>plugins</i> y <i>consumers</i> .
8100	Kong Status / metrics	<i>Endpoint /metrics</i> para Prometheus.
8081	CientesApi	Acceso directo de diagnóstico al <i>backend</i> de clientes.
8082	CuentasApi	Acceso directo de diagnóstico al <i>backend</i> de cuentas.
5672	RabbitMQ AMQP	Canal de mensajería interno para publicación y consumo.
15672	RabbitMQ Management	Interfaz web de administración del <i>broker</i> .
9090	Prometheus	Interfaz web y API de consulta de métricas.
3000	Grafana	Interfaz web de <i>dashboards</i> .

Tabla T.5. Puertos expuestos por la PoC

```
# Comprobar puertos en uso antes del despliegue
```

```
sudo ss -tulpn | egrep ':8000|:8001|:8100|:8081|:8082|:5672|:15672|:9090|:3000'
```

5. Obtención del producto y estructura esperada

El producto se entrega como repositorio GitHub público y/o como paquete comprimido en el espacio de entrega de la asignatura. En ambos casos, el resultado esperado es disponer del código fuente y de los ficheros de despliegue en una carpeta local de trabajo.

```
cd ~
git clone https://github.com/amartosg/tfg-plataforma-segura-apis.git
cd tfg-plataforma-segura-apis
```

La estructura exacta puede variar en la versión final del repositorio, pero el evaluador debe encontrar al menos los elementos indicados en la Tabla T.6.

Elemento	Finalidad
docker-compose.yml	Descriptor principal de servicios Docker.
env.example.txt o .env.example	Plantilla de variables de entorno sin secretos reales.
get_token.sh	Script auxiliar para solicitar token a Entra ID.
benchmark_gateway_overhead.sh	Script opcional para reproducir el <i>benchmark</i> de <i>overhead</i> .
CientesApi/	Código fuente de la API REST de clientes.
CuentasApi/	Código fuente de la API REST de cuentas.
ClienteCreadoConsumer/	Worker Service .NET para consumo de eventos RabbitMQ.
prometheus/	Configuración prometheus.yml o ficheros equivalentes.
grafana/	<i>Provisioning</i> o JSON del <i>dashboard</i> de Grafana, si se incluye.
scripts/	Scripts auxiliares de configuración de Kong y validación.

Tabla T.6. Estructura esperada del producto entregado

```
# Validación rápida de estructura
ls -la
find . -maxdepth 2 -type f | sort | head -50
```

6. Configuración de variables de entorno y secretos

La PoC separa la configuración sensible del código fuente. El fichero .env debe crearse a partir de la plantilla proporcionada y mantenerse fuera del repositorio mediante .gitignore.

```
# Crear el fichero local de entorno a partir de la plantilla disponible
cp env.example.txt .env # si la plantilla se llama env.example.txt
# o bien:
# cp .env.example .env

nano .env
```

Variable	Uso
ENTRA_TENANT_ID	Identificador del tenant de Microsoft Entra ID.
ENTRA_CLIENT_ID	Client ID de la aplicación cliente que solicita el token.
ENTRA_CLIENT_SECRET	Secreto de la aplicación cliente. No debe versionarse ni capturarse en claro.

Variable	Uso
ENTRA_RESOURCE_APP_ID	Application ID de la aplicación recurso que representa la API.
ENTRA_SCOPE / SCOPE	Normalmente api://<RESOURCE_APP_ID>/.default. Se recomienda ENTRA_SCOPE para mantener coherencia con el Anexo U de configuración, SCOPE se mantiene si el script del repositorio usa ese nombre.
KONG_DATABASE	Postgres, al usar Kong con PostgreSQL como almacén de configuración.
RABBITMQ_DEFAULT_USER	Usuario de administración del <i>broker</i> para laboratorio.
RABBITMQ_DEFAULT_PASS	Contraseña local de RabbitMQ. Debe ser de laboratorio y no reutilizarse.
GF_SECURITY_ADMIN_USER	Usuario administrador de Grafana si se parametriza en Compose.
GF_SECURITY_ADMIN_PASSWORD	Contraseña local de Grafana. No debe publicarse en el repositorio.

Tabla T.7. Variables de entorno relevantes

Buenas prácticas: no incluir el fichero .env en la memoria, anexos, capturas o repositorio. En las evidencias, ocultar client secrets, tokens y cualquier valor que permita autenticación.

6.1. Comprobación de obtención de token

Una vez configurado .env, puede verificarse la emisión del token. La salida no debe publicarse completa, basta con comprobar que existe un access_token y que el JWT contiene la audiencia y los roles esperados.

```
chmod +x get_token.sh
./get_token.sh

# Alternativa manual si se desea diagnosticar la llamada a Entra ID:
source .env
TOKEN=$(curl -s -X POST \
  "https://login.microsoftonline.com/${ENTRA_TENANT_ID}/oauth2/v2.0/token" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "client_id=${ENTRA_CLIENT_ID}" \
  -d "scope=api://${ENTRA_RESOURCE_APP_ID}/.default" \
  -d "client_secret=${ENTRA_CLIENT_SECRET}" \
  -d "grant_type=client_credentials" | jq -r '.access_token')

printf "Token recibido: %.30s...\n" "$TOKEN"
```

7. Instalación y arranque de la PoC con Docker Compose

El despliegue se realiza desde la raíz del repositorio. Antes de levantar los servicios, se recomienda partir de un estado limpio para evitar residuos de pruebas anteriores.

```
# Desde la raíz del repositorio
cd ~/tfg-plataforma-segura-apis
```

```
# Limpieza segura de ejecuciones anteriores
# Advertencia: elimina contenedores y volúmenes de esta PoC.
docker compose down -v --remove-orphans

# Construcción de imágenes locales
docker compose build --no-cache

# Arranque en segundo plano
docker compose up -d

# Estado de los servicios
docker compose ps
```

La Tabla T.8 resume el estado esperado tras el arranque inicial.

Servicio	Estado esperado	Comprobación
kong-database	Up / healthy	PostgreSQL disponible para Kong.
kong-migrations	Exited 0	Job de inicialización completado correctamente.
kong	Up	Gateway disponible en 8000, 8001 y 8100.
clientes-api	Up	API de clientes disponible dentro de la red Docker y diagnóstico en 8081.
cuentas-api	Up	API de cuentas disponible dentro de la red Docker y diagnóstico en 8082.
rabbitmq	Up	Broker AMQP y Management UI disponibles.
cliente-creado-consumer	Up	Worker conectado a RabbitMQ y escuchando la cola.
prometheus	Up	Prometheus disponible en 9090.
grafana	Up	Grafana disponible en 3000.

Tabla T.8. Estado esperado de contenedores tras docker compose up -d

```
# Logs generales de diagnóstico
docker compose logs --tail=80 kong
docker compose logs --tail=80 clientes-api
docker compose logs --tail=80 cuentas-api
docker compose logs --tail=80 rabbitmq
docker compose logs --tail=80 cliente-creado-consumer
```

8. Inicialización de Kong Gateway y publicación de APIs

Kong Gateway requiere que se declaren servicios, rutas, consumers y plugins. Si el repositorio final incluye scripts de automatización, se recomienda utilizarlos para evitar errores manuales. En caso contrario, la configuración puede reproducirse mediante la Admin API de Kong en el puerto 8001.

```
# Opción recomendada si existen scripts de configuración en el repositorio
find scripts -maxdepth 1 -type f -name "*.sh" -print -exec chmod +x {} \;
# Ejemplos de nombres habituales. Ejecutar el que exista en la versión final:
# ./scripts/configure_kong.sh
# ./scripts/setup_kong.sh
# ./scripts/init_kong.sh
```


La configuración mínima que debe quedar registrada en Kong se resume en la Tabla T.9.

Objeto Kong	Valor esperado	Función
clientes-service	http://clientes-api:80	Servicio <i>upstream</i> para ClientesApi.
clientes-route	/clientes	Ruta pública expuesta por el plano de datos de Kong.
cuentas-service	http://cuentas-api:80	Servicio <i>upstream</i> para CuentasApi.
cuentas-route	/cuentas	Ruta pública expuesta por el plano de datos de Kong.
Plugin jwt	Servicios protegidos	Valida JWT RS256 emitidos por Entra ID.
Plugin rate-limiting	Servicios protegidos	Limita el tráfico por <i>consumer</i> y devuelve 429 al superar el umbral.
Plugin prometheus	Global	Expone métricas del <i>gateway</i> en /metrics.
Consumer	appid / cliente Entra ID	Vincula el token entrante con la credencial JWT registrada en Kong.

Tabla T.9. Configuración mínima esperada en Kong Gateway

Comandos de consulta para confirmar que Kong contiene los objetos de configuración:

```
curl -s http://localhost:8001/services | jq '.data[].name'
curl -s http://localhost:8001/routes | jq '.data[] | {name, paths, strip_path}'
curl -s http://localhost:8001/plugins | jq '.data[] | {name, enabled}'
curl -s http://localhost:8001/consumers | jq '.data[].username'
```

Punto crítico de seguridad: si Entra ID rota la clave pública utilizada para firmar los JWT, debe actualizarse la credencial RS256 registrada en Kong. Un error de firma se manifestará como HTTP 401 aunque el token se haya emitido correctamente.

9. Verificación funcional de la instalación

La verificación funcional se divide en pruebas de infraestructura, pruebas directas de *backend* y pruebas extremo a extremo a través de Kong Gateway.

9.1. Pruebas de infraestructura

```
# Kong Status API
curl -i http://localhost:8100/status

# Métricas de Kong para Prometheus
curl -s http://localhost:8100/metrics | head -20

# Prometheus readiness
curl -i http://localhost:9090/-/ready

# Backends directos de diagnóstico
curl -i http://localhost:8081/health
curl -i http://localhost:8082/health
```

9.2. Pruebas extremo a extremo vía Kong

Las llamadas funcionales deben realizarse contra el plano de datos de Kong (puerto 8000), no directamente contra los *backends*. De esta forma se valida que la seguridad y el gobierno de APIs pasan por el *gateway*.

```
# Cargar el token en una variable de sesión. No mostrarlo completo en pantalla.
source .env
TOKEN=$(./get_token.sh | tail -1)

# Consulta de clientes a través de Kong
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8000/clientes

# Consulta de cuentas a través de Kong
curl -i -H "Authorization: Bearer $TOKEN" http://localhost:8000/cuentas

# Ejemplo de consulta de saldo; ajustar el identificador a los datos simulados.
curl -i \
  -H "Authorization: Bearer $TOKEN" \
  http://localhost:8000/cuentas/ACC-001/saldo
```

Escenario	Prueba	Resultado esperado	Interpretación
Sin token	GET /clientes	401 Unauthorized	Kong bloquea la petición por ausencia de credencial.
Token inválido	GET /clientes	401 Unauthorized	Kong rechaza el JWT por firma, formato o vigencia.
Token válido con rol	GET /clientes	200 OK	La petición supera <i>gateway</i> y autorización de <i>backend</i> .
Token válido sin rol suficiente	GET /cuentas	403 Forbidden	El <i>backend</i> deniega la operación por autorización diferencial.
Exceso de peticiones	Más de 10 llamadas/minuto	429 Too Many Requests	Kong aplica <i>rate limiting</i> por <i>consumer</i> .

Tabla T.10. Pruebas funcionales mínimas de instalación

10. Verificación de observabilidad y mensajería asíncrona

10.1. Prometheus

Prometheus debe tener configurado un *job* de *scraping* contra Kong en el puerto 8100. Tras generar tráfico contra el *gateway*, las métricas deben aparecer en la interfaz web de Prometheus.

```
# Comprobar targets desde la interfaz web:
# http://localhost:9090/targets

# Consultas PromQL útiles en la interfaz de Prometheus:
kong_nginx_requests_total
rate(kong_nginx_requests_total[1m])
```

```
kong_http_requests_total
sum by (code)(rate(kong_http_requests_total[5m]))
kong_latency_ms
kong_datastore_reachable
up{job="kong"}
```

10.2. Grafana

Grafana debe estar disponible en <http://localhost:3000>. El datasource de Prometheus debe apuntar a <http://prometheus:9090> dentro de la red Docker. Si el dashboard no muestra datos, conviene generar varias peticiones contra Kong y esperar al menos un intervalo de scraping. El dashboard final debe incluir los seis paneles validados en la memoria: tasa de peticiones, respuestas HTTP por código, total de respuestas por código, errores técnicos 5xx, porcentaje de peticiones rechazadas o limitadas y estado del datastore.

- Entrar en Grafana con el usuario y contraseña definidos para el laboratorio o con las credenciales por defecto documentadas en el repositorio.
- Verificar que existe el *datasource* Prometheus y que el botón de prueba devuelve conexión correcta.
- Abrir el *dashboard* de la PoC y comprobar paneles de tráfico, códigos HTTP, rechazos 401/403/429, errores 5xx y estado del *datastore*.

10.3. RabbitMQ y Worker Service

La mensajería asíncrona se valida creando un cliente mediante la API de clientes. La API publica el evento ClienteCreado en RabbitMQ y el Worker Service lo consume, registrando el procesamiento en sus logs.

```
# Acceso web a RabbitMQ Management:
# http://localhost:15672

# Logs del consumidor asíncrono
docker compose logs -f cliente-creado-consumer

# Alta de cliente vía Kong. Ajustar payload al contrato final de la API.
curl -i -X POST http://localhost:8000/clientes \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"nombre":"Cliente","apellido":"Demo","segmento":"Banca Privada"}'
```

Elemento	Evidencia esperada
RabbitMQ UI	El <i>broker</i> está accesible en 15672 y muestra la cola cliente-creado.
Alta de cliente	La API devuelve respuesta 200/201 según contrato final.
Cola	El mensaje se publica de forma persistente y se consume posteriormente.
Worker	Los logs del contenedor cliente-creado-consumer muestran el evento procesado.

Tabla T.11. Evidencias mínimas del flujo asíncrono

11. Parada, limpieza y reinstalación desde cero

La reproducibilidad de la PoC exige poder detener, limpiar y levantar de nuevo el entorno con un procedimiento controlado.

```
# Parada sin eliminar volúmenes
```

```

docker compose stop

# Arranque posterior
docker compose start

# Parada y eliminación de contenedores, redes y volúmenes de la PoC
docker compose down -v --remove-orphans

# Limpieza opcional de imágenes no usadas
# Ejecutar solo si se desea liberar espacio en la VM.
docker system prune -f

# Reinstalación desde cero
docker compose build --no-cache
docker compose up -d
docker compose ps

```

Advertencia: `docker compose down -v` elimina los volúmenes de PostgreSQL, RabbitMQ, Prometheus y Grafana asociados a la PoC. Es el comportamiento adecuado para validar reinstalación limpia, pero no debe ejecutarse si se desean conservar evidencias operativas.

12. Incidencias habituales y resolución

Incidencia	Causa probable	Acción recomendada
Docker permission denied	El usuario no pertenece al grupo docker o no se ha reiniciado la sesión.	Ejecutar <code>sudo usermod -aG docker \$USER</code> y reiniciar sesión o VM.
Puerto ocupado	Otro proceso utiliza 8000, 8001, 3000, 9090 o 15672.	Localizar con <code>ss -tuln</code> y liberar o modificar temporalmente el mapeo de puertos.
Kong no arranca	PostgreSQL no está listo o las migraciones fallaron.	Revisar <code>docker compose logs kong-database kong-migrations kong</code> y repetir con <code>down -v</code> .
401 con token aparentemente válido	Clave pública RS256 registrada en Kong no coincide con el kid usado por Entra ID o <code>key_claim_name</code> incorrecto.	Actualizar la credencial JWT de Kong con la clave pública vigente y revisar <code>appid</code> .
403 Forbidden	El token no contiene el app rol requerido para la operación.	Revisar asignación de app roles en Entra ID y consentimiento de administrador.
429 Too Many Requests	Rate limiting activado y umbral superado.	Esperar el tiempo indicado por <code>Retry-After</code> o ajustar el umbral para pruebas.
Grafana sin datos	No hay tráfico reciente o Prometheus no scrapea Kong.	Generar tráfico, revisar <code>http://localhost:9090/targets</code> y comprobar <code>datasource</code> .
RabbitMQ sin mensajes	La API no publica el evento o el <i>worker</i> lo consume inmediatamente.	Revisar logs de <code>clientes-api</code> , <code>rabbitmq</code> y <code>cliente-creado-consumer</code> .

Tabla T.12. Incidencias habituales durante la instalación

13. Checklist de evidencias para la entrega final

La siguiente lista permite recopilar capturas y salidas de terminal suficientes para acreditar que la instalación del producto es reproducible y verificable.

Código	Evidencia	Contenido mínimo
E-INS-01	Captura de la configuración de la VM	VMware/Ubuntu con CPU, RAM, disco y red NAT.
E-INS-02	Versiones de Docker	Salida de docker --version y docker compose version.
E-INS-03	Repositorio clonado	ls -la mostrando docker-compose.yml y carpetas de APIs.
E-INS-04	Variables configuradas	Captura parcial de .env con secretos ocultos.
E-INS-05	Contenedores levantados	docker compose ps con servicios Up y kong-migrations Exited 0.
E-INS-06	Kong Admin API	curl a /services, /routes y /plugins mostrando configuración.
E-INS-07	Token de Entra ID	Evidencia de obtención de token sin mostrarlo completo.
E-INS-08	Prueba 200 vía Kong	GET /clientes o /cuentas con token válido.
E-INS-09	Prueba 401	Llamada sin token o con token inválido.
E-INS-10	Prueba 403	Llamada con token sin rol suficiente.
E-INS-11	Prueba 429	Rate limiting superado y cabecera Retry-After.
E-INS-12	Prometheus	Targets UP y métrica kong_nginx_requests_total.
E-INS-13	Grafana	Dashboard con paneles y datos generados por tráfico real.
E-INS-14	RabbitMQ	Cola cliente-creado y logs del Worker Service procesando evento.

Tabla T.13. Checklist de evidencias de instalación

14. Criterios de aceptación de la instalación

La instalación se considera aceptada cuando el entorno puede reproducirse desde una máquina Linux limpia, los servicios se levantan sin errores críticos, las APIs se consumen exclusivamente a través de Kong Gateway y las evidencias funcionales, de seguridad, observabilidad y mensajería quedan registradas sin exponer secretos. Estos criterios permiten diferenciar una instalación meramente ejecutada de una instalación verificable y defendible en la entrega final.

Criterio	Descripción	Evidencia asociada
CA-INS-01	La máquina virtual cumple los recursos mínimos definidos y dispone de Docker Engine y Docker Compose v2 operativos.	E-INS-01, E-INS-02
CA-INS-02	El repositorio o paquete entregado contiene docker-compose.yml, plantillas de entorno, APIs, scripts y configuración de observabilidad.	E-INS-03
CA-INS-03	El fichero .env se genera desde plantilla, queda fuera del repositorio y no se muestra con secretos en claro.	E-INS-04
CA-INS-04	Los contenedores principales arrancan en estado Up y el job de migraciones de Kong finaliza correctamente.	E-INS-05

Criterio	Descripción	Evidencia asociada
CA-INS-05	Kong Gateway queda inicializado con servicios, rutas, plugins JWT/rate-limiting/prometheus y <i>consumers</i> .	E-INS-06
CA-INS-06	La validación funcional vía Kong devuelve respuestas 200 para casos autorizados y 401/403/429 para escenarios de control.	E-INS-08 a E-INS-11
CA-INS-07	Prometheus y Grafana muestran métricas generadas por tráfico real y RabbitMQ procesa el evento ClienteCreado.	E-INS-12 a E-INS-14

Tabla T.14. Criterios de aceptación de la instalación

15. Trazabilidad instalación-validación-evidencia

La siguiente matriz relaciona las actividades de instalación con los componentes de la PoC, el tipo de validación requerido y la evidencia que debe conservarse. Su objetivo es facilitar la revisión del producto por parte del tribunal y demostrar que el anexo no se limita a enumerar comandos, sino que documenta una instalación verificable y alineada con los objetivos del TFG.

Actividad	Componente afectado	Validación	Evidencia
Preparación de la VM	Ubuntu, red NAT, recursos de cómputo	Sistema base disponible y puertos libres	E-INS-01
Instalación de Docker	Docker Engine y Compose	Versiones instaladas y usuario con permisos adecuados	E-INS-02
Obtención del producto	Repositorio GitHub o paquete entregado	Estructura del producto completa	E-INS-03
Configuración de entorno	.env y variables de Entra ID/RabbitMQ/Grafana	Secretos ocultos y fichero no versionado	E-INS-04
Arranque de contenedores	Kong, APIs, RabbitMQ, Prometheus y Grafana	Servicios Up y migraciones completadas	E-INS-05
Publicación en Kong	Servicios, rutas, consumers y plugins	Admin API devuelve objetos esperados	E-INS-06
Validación de identidad	Microsoft Entra ID y JWT	Token emitido y usado sin exponerlo completo	E-INS-07
Validación funcional y de seguridad	CientesApi, CuentasApi y Kong	Respuestas 200, 401, 403 y 429 controladas	E-INS-08 a E-INS-11
Validación de observabilidad y mensajería	Prometheus, Grafana, RabbitMQ y Worker	Métricas, <i>dashboard</i> , cola y logs del consumidor	E-INS-12 a E-INS-14

Tabla T.15. Trazabilidad instalación-validación-evidencia

16. Limitaciones y cierre operativo

La instalación documentada responde a una PoC académica y no debe interpretarse como una guía de despliegue productivo. El entorno local permite validar arquitectura, seguridad de APIs, integración entre componentes y observabilidad, pero no incorpora capacidades propias de explotación empresarial como alta disponibilidad, balanceo multiinstancia, gestión centralizada de secretos, *hardening* de red corporativo, WAF, rotación automática de claves, monitorización avanzada de infraestructura o despliegue gobernado en Kubernetes gestionado.

Desde el punto de vista de seguridad, las evidencias de instalación deben capturarse con especial cuidado: los tokens JWT deben mostrarse truncados, los *client secrets* no deben aparecer en pantalla, las contraseñas locales deben ocultarse y el fichero `.env` no debe entregarse. Esta restricción no reduce la reproducibilidad del anexo, ya que la estructura de variables, comandos y pruebas permite validar el producto sin divulgar información sensible.

Cierre del anexo: si se completan las evidencias E-INS-01 a E-INS-14 y se cumplen los criterios CA-INS-01 a CA-INS-07, la PoC queda instalada, operativa y lista para ser validada mediante el Anexo V de acceso y uso del producto. La instalación acredita que el producto entregado es reproducible, que la plataforma de APIs puede arrancar en entorno local y que los componentes principales quedan disponibles para las pruebas funcionales, de seguridad y de observabilidad descritas en la memoria.

17. Referencias bibliográficas

Docker Inc. (s. f.). Docker Compose. Docker Docs. <https://docs.docker.com/compose/>

Docker Inc. (s. f.). Install Docker Engine on Ubuntu. Docker Docs. <https://docs.docker.com/engine/install/ubuntu/>

Docker Inc. (s. f.). Linux post-installation steps for Docker Engine. Docker Docs. <https://docs.docker.com/engine/install/linux-postinstall/>

Grafana Labs. (s. f.). Grafana documentation. <https://grafana.com/docs/grafana/latest/>

Kong Inc. (s. f.). Kong Gateway documentation. <https://docs.konghq.com/gateway/>

Kong Inc. (s. f.). JWT authentication plugin. Kong Docs. <https://developer.konghq.com/plugins/jwt/>

Kong Inc. (s. f.). Prometheus plugin. Kong Docs. <https://developer.konghq.com/plugins/prometheus/>

Kong Inc. (s. f.). Rate Limiting plugin. Kong Docs. <https://developer.konghq.com/plugins/rate-limiting/>

Microsoft. (s. f.). Application roles. Microsoft Learn. <https://learn.microsoft.com/entra/identity-platform/howto-add-app-roles-in-apps>

Microsoft. (2026, 30 de enero). OAuth 2.0 client credentials flow on the Microsoft identity platform. Microsoft Learn. <https://learn.microsoft.com/entra/identity-platform/v2-oauth2-client-creds-grant-flow>

Microsoft. (s. f.). ASP.NET Core documentation. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/>

Open Worldwide Application Security Project. (2023). OWASP API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/0x00-header/>

Prometheus Authors. (s. f.). Prometheus documentation. <https://prometheus.io/docs/>

RabbitMQ. (s. f.). RabbitMQ documentation. <https://www.rabbitmq.com/docs>